High-Performance Spreadsheet Grid Rendering: Architectures for Fast Cell Geometry

To render spreadsheet cell dimensions—grid lines, cell backgrounds, custom borders, resized rows and columns, frozen panes, and merged cells—at 60 to 120 frames per second (FPS) without formula evaluation or text layout, you must decouple **logical grid geometry** from **physical pixel rendering**[1]. High-performance spreadsheet engines achieve sub-16ms frame budgets by combining two core architectural mechanisms: (1) replacing per-cell Document Object Model (DOM) nodes with **batched immediate-mode HTML5 2D Canvas paths**, **WebGL/WebGPU shaders**, or **strictly pooled 2D virtualized DOM rows**, and (2) resolving variable row heights, column widths, and merged spans via $O(\log N)$ **or sparse run-length encoded (RLE) spatial trees** rather than $O(N)$ linear array scans[1]. Finally, because web browsers clamp native CSS element heights and scroll coordinates to fixed-point ceilings between 16.77 million and 33.55 million pixels, scrolling across a million-row sheet requires **virtual scroll downscaling**, **sticky runway recentering**, or a **custom stateless scroll pane**[8]. These techniques are necessary because standard HTML tables fail under both draw-call volume and coordinate scale[1]. Even a modest viewport displaying 50 visible rows by 50 columns mounts 2,500 individual DOM elements (<td> or), each incurring CSS style recalculation, bounding-box layout (reflow), and GPU compositor overhead during panning[1]. When a workbook scales to Microsoft Excel's rigid grid of 1,048,576 rows by 16,384 columns (over 17.18 billion cells) or Google Sheets' 20-million-cell shared workbook pool (supporting up to 18,278 columns, terminating at column ZZZ), mutating thousands of DOM nodes per scroll tick overwhelms the browser rendering pipeline[4].

In practice, production spreadsheet UIs select from three rendering tiers depending on scale and interaction requirements[1]:

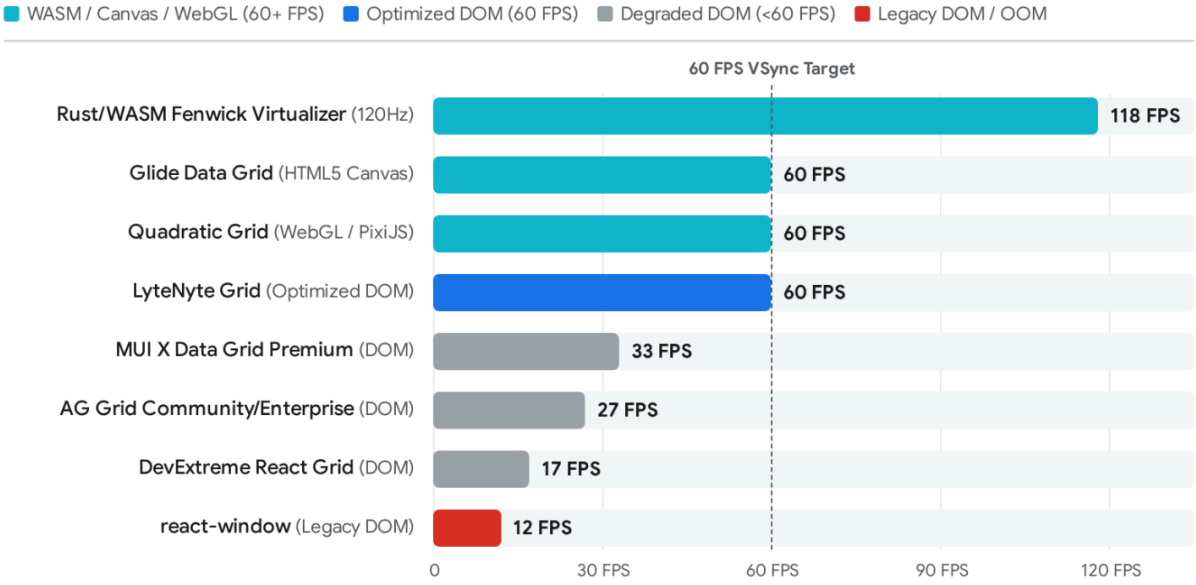
1. **Immediate-Mode HTML5 2D Canvas & OffscreenCanvas (Industry Standard):** Used by Google Sheets, Glide Data Grid, and the open-source Univer SDK, this architecture collapses thousands of cell elements into a single <canvas> bitmap[2]. By batching all visible horizontal and vertical grid lines into a single beginPath() and stroke() call ($R + C + 2$ line segments instead of $4 \times R \times C$ individual cell borders) and shifting rasterization to a Web Worker via OffscreenCanvas, Canvas 2D grids reported in developer analyses sustain 60 FPS across 10 million rows with a flat memory footprint[2].
2. **WebGL / WebGPU Batched Geometry & Procedural Shaders (Maximum Throughput):** Used by game-engine-style spreadsheets such as Quadratic (built on PixiJS) and infinite-canvas tools, WebGL offloads rasterization to custom GPU shaders[1]. Uniform background grids render in a **single draw call** using a 4-vertex full-screen quad and a screen-space fragment shader (fwidth anti-aliasing), while variable-sized cells

batch extruded line quads and background triangles into **15×30 cell spatial hashes**, according to Quadratic's engineering documentation, rendering 100,000 visible cells in 10 to 50 GPU draw calls[1].

3. **Hyper-Optimized 2D Virtualized DOM (When Native DOM Is Required):** When application constraints require DOM cells, sustaining 60 FPS demands two-axis virtualization with DOM node recycling, row-level GPU transform: translateY() positioning (avoiding per-cell compositor layer explosion), and explicit pixel sizing[4]. In a vendor benchmark published by 1771 Technologies (creators of LyteNyte Grid) on an Apple M4 MacBook Air (Chrome v149), LyteNyte Grid maintained 60 FPS at 1 million rows, whereas legacy or heavier DOM architectures—including MUI X Data Grid Premium (33 FPS at 1M rows), AG Grid (27 FPS at 1M rows), DevExtreme (17 FPS), and Handsontable (which crashed with out-of-memory errors above ~150,000 rows)—degraded under layout and compositor commit pressure[13].

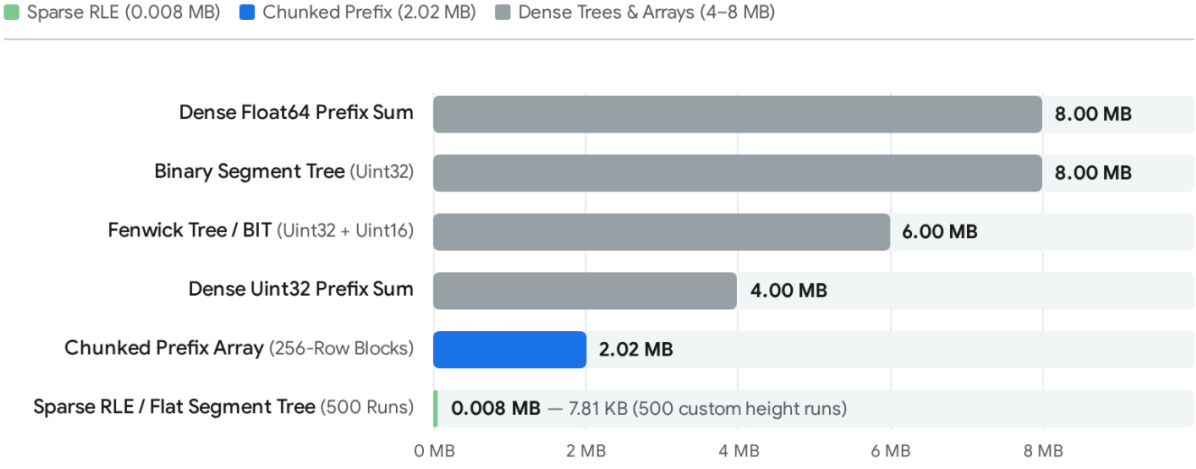
Canvas, WebGL, and Optimized Engines Sustain 60+ FPS While Sparse Trees Cut Memory by 99.9%

PANEL A: VERTICAL SCROLLING THROUGHPUT AT 1,000,000 ROWS (FPS)



OOM CRASH **Handsontable (DOM)** crashes with Out-Of-Memory (OOM) errors on datasets exceeding ~150,000 rows (averaging 29 FPS below that threshold).

PANEL B: MEMORY FOOTPRINT FOR INDEXING 1,048,576 ROW HEIGHTS (MB)



Top panel shows vertical scrolling frame rates at 1,000,000 rows (with 60Hz vsync caps on browser benchmarks and 120Hz uncapped on WASM Fenwick tests); Handsontable fails with out-of-memory errors above 150,000 rows. Bottom panel compares RAM required to index 1,048,576 variable row heights, demonstrating how sparse run-length encoded (RLE) flat segment trees reduce memory from 8.00 MB to 0.008 MB (7.81 KB) when custom row heights occur in 500 contiguous runs.

Data sources: Quadratic HQ, BrightCoding (Glide Data Grid), 1771 Technologies, and Rust/WASM Virtualization Benchmarks.

1. Rendering Backends: DOM vs. HTML5 2D Canvas vs. WebGL/WebGPU

Selecting the rendering backend dictates your frame budget, memory ceiling, and zoom capabilities. While all three approaches rely on **viewport culling**—computing and drawing only the cells that intersect the visible viewport—they differ fundamentally in how they submit cell geometry to the browser's graphics pipeline[1].

Architectural Dimension	2D Virtualized DOM (e.g., LyteNyte, AG Grid)	HTML5 2D Canvas / OffscreenCanvas (e.g., Google Sheets, Glide, Univer)	WebGL / WebGPU Shaders & Meshes (e.g., Quadratic)
Active Scene Primitives	1,500–3,000 live HTML nodes (viewport + overscan)[4]	1 <canvas> element (or 2–4 layered/pinned canvases); 0 cell DOM nodes[2]	1 WebGL <canvas> context; 4-vertex quad or batched Float32Array vertex buffers[1]
Grid Line Draw Mechanism	CSS border-right / border-bottom on each cell (thousands of box borders)[1]	Single batched path (ctx.beginPath(), $R + C + 2$ moveTo/lineTo calls, 1 ctx.stroke())[6]	Dedicated Line Shader (extruded triangle strips) or fragment shader fwidth() math[1]
Per-Frame CPU Bottleneck	Style recalculation, DOM tree mutations, layout reflows, compositor layer commits[1]	Immediate-mode 2D context state changes (strokeStyle, fillStyle) and rasterization[2]	Minimal on main thread; vertex buffer generation offloaded to Web Worker[1]
GPU Draw Calls per Frame	Hundreds to thousands (unbatched DOM boxes with unique styles)[1]	Batched by path/color via browser's 2D canvas backend[2]	**1 draw call** (uniform shader grid) to **10–50 draw calls** (100K visible cells)[1]
Continuous Zoom	Poor (CSS scale blurs borders or	Moderate (requires manual DPR	**Native 60 FPS** (matrix uniform

(0.01x–10x)	triggers massive DOM node counts when zooming out)[1]	rescaling and culling adjustments at extreme zoom-out)[22]	update + spatial hash LOD culling + fwidth derivatives)[1]
Reported 1M-Row Scroll FPS	12–33 FPS (standard DOM grids); up to 60 FPS in lean DOM engines (LyteNyte)[13]	**60 FPS** reported up to 10,000,000 rows with sub-16ms refresh cycles[2]	**60–118 FPS** reported across millions of cells in WebGL/WASM tests[1]

Tier 1: HTML5 2D Canvas & OffscreenCanvas Architecture

For dedicated spreadsheet applications—including Google Sheets, Glide Data Grid, and Univer—HTML5 2D Canvas is the dominant architectural choice[2]. Because a spreadsheet grid without cell contents consists entirely of orthogonal line segments, filled rectangles, and selection outlines, rendering it via the DOM forces the browser to parse HTML/CSS, construct a Render Tree, run layout reflows, and composite thousands of identical boxes[4]. Canvas bypasses the DOM layout engine, painting geometry directly onto a hardware-accelerated surface[2].

To maximize Canvas 2D geometry throughput, your rendering loop should implement four specific optimizations:

1. Path Batching by Graphic State

A frequent bottleneck in naive Canvas spreadsheet renderers is iterating over every visible cell (r, c) and calling ctx.strokeRect(x, y, width, height) or ctx.fillRect(x, y, width, height) inside a nested loop. In an 80-row by 20-column viewport (1,600 visible cells), calling strokeRect per cell issues 1,600 individual draw commands and overdraws every interior grid line twice (once for each adjacent cell)[18].

Instead, decouple **background fills**, **standard grid lines**, and **custom borders** into separate batched passes. First, paint the entire viewport background with a single ctx.fillRect(0, 0, viewportWidth, viewportHeight) using the default sheet fill[22]. Only issue cell-level ctx.fillRect calls for cells carrying non-default background colors, grouping those rectangles by color string so ctx.fillStyle mutates only once per unique color. Second, execute a single **batched grid line pass**: because a rectangular grid of R visible rows and C visible columns is bounded by $R + 1$ continuous horizontal lines across the viewport width and $C + 1$ continuous vertical lines across the viewport height, you can call ctx.beginPath() once, loop through the $R + 1$ row Y-coordinates with ctx.moveTo(0, y) and ctx.lineTo(viewportWidth, y), loop through the $C + 1$ column X-coordinates with ctx.moveTo(x, 0) and ctx.lineTo(x, viewportHeight), and invoke ctx.stroke() **once** at the end[6]. For an 80×20 viewport, this

collapses 1,600 rectangle strokes into 102 line segments in a single GPU rasterization pass.

2. Sub-Pixel Alignment (+0.5px) and devicePixelRatio Scaling

In the HTML5 Canvas coordinate system, integer coordinates (x, y) align with the infinitely thin mathematical boundaries *between* physical pixels rather than the center of a pixel[25]. When you draw a lineWidth = 1 vertical grid line at integer coordinate x = 25, the stroke expands 0.5px to either side, spanning from 24.5 to 25.5[25]. Because physical display pixels cannot be partially illuminated, the browser anti-aliases the stroke across both pixel column 24 and pixel column 25 at half opacity, creating a blurry 2-pixel-wide line[25].

To render crisp 1-physical-pixel grid lines across standard 1x displays and HiDPI/Retina monitors (window.devicePixelRatio of 1.5, 2, or 3)[23]:

1. **Scale the Physical Backing Store:** Set the canvas buffer dimensions to canvas.width = Math.round(logicalWidth * dpr) and canvas.height = Math.round(logicalHeight * dpr), while locking the CSS layout size to canvas.style.width = $logicalWidthpxand`canvas.style.height = `{logicalHeight}px[21].
2. **Snap to Physical Integers and Offset by 0.5 Pixels:** Rather than relying on ctx.scale(dpr, dpr) with fractional logical coordinates (which can still land between physical pixels under fractional scaling like dpr = 1.5), convert logical cell boundary coordinates into physical device pixels, snap to an integer via Math.floor(logicalCoord * dpr), and add 0.5 physical pixels when stroking a 1-device-pixel line[6]:

$$x_{\text{physical}} = \lfloor x_{\text{logical}} \times \text{DPR} \rfloor + 0.5$$

This centers the stroke at 25.5 (spanning 25.0 to 26.0), illuminating a single column of physical pixels without blur[25].

3. Frozen Panes, Headers, and Scissor Clipping

A spreadsheet with frozen rows (pinned top) and frozen columns (pinned left) divides the grid into four coordinate quadrants—Top-Left Frozen Corner (static in X and Y), Top Frozen Rows (scrolls in X, static in Y), Left Frozen Columns (static in X, scrolls in Y), and Main Scrollable Body (scrolls in X and Y)—alongside the Row Header and Column Header rulers (SpreadsheetRowHeader and SpreadsheetColumnHeader)[2].

In layered Canvas architectures such as Univer's engine-render, an Engine manages the <canvas> instance while a Scene coordinates multiple Viewport regions backed by a pre-calculated layout cache (SheetSkeleton)[3]. Before drawing each quadrant on a shared canvas, isolate its rectangular region using ctx.save(), ctx.beginPath(), ctx.rect(vpX, vpY, vpWidth, vpHeight), and ctx.clip(), draw the visible cells for that quadrant's scroll offset, and call ctx.restore() so partially scrolled cells never bleed across freeze lines or over row/column headers[2].

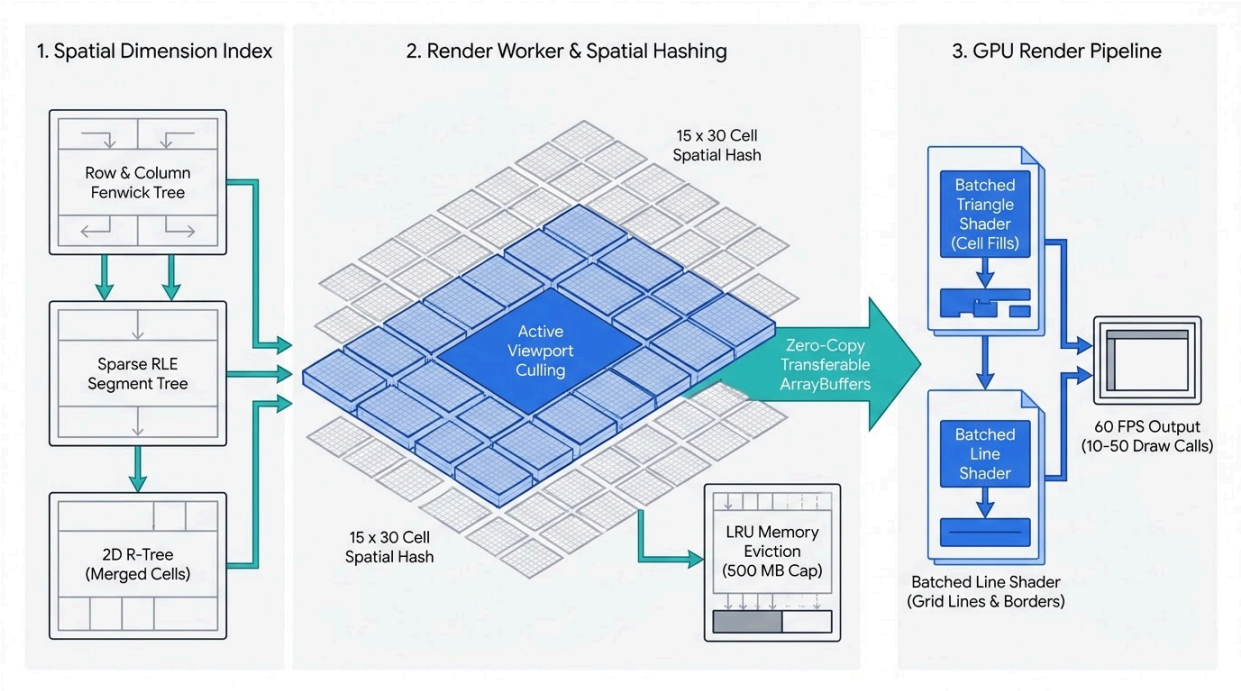
4. Offloading Rasterization via OffscreenCanvas

Even when Canvas draw calls are batched, heavy work on the main browser thread—such as complex toolbar updates, React reconciliations, or data ingestion—can delay

requestAnimationFrame callbacks and freeze scrolling[33]. Calling const offscreen = htmlCanvas.transferControlToOffscreen() and transferring the surface to a dedicated Web Worker via zero-copy worker.postMessage({ canvas: offscreen }, [offscreen]) decouples rendering from the main DOM thread[14]. The worker processes scroll delta messages and executes its own worker.requestAnimationFrame loop, keeping grid panning responsive even when the main thread is blocked[33]. OffscreenCanvas is supported in Chrome 69+, Edge 79+, Firefox 105+, and Safari 16.4+[14].

Tier 2: WebGL / WebGPU Shaders & Spatial Hashing

When a spreadsheet UI requires **continuous camera zooming** (e.g., smoothly zooming from 0.01x to 10x across millions of cells) or ultra-low GPU draw calls, immediate-mode 2D Canvas encounters CPU path-rasterization bottlenecks[1]. Treating the spreadsheet grid like a 2D game engine using WebGL or WebGPU solves this through procedural fragment shaders and spatial hash meshes[1].



1. Procedural Screen-Space Fragment Shaders (Zero Line Geometry)

For uniform grid layers—or for rendering an infinite background reference grid—you do not need to generate vertex buffers for individual lines[15]. Instead, submit a single full-screen quad of 4 vertices in clip space ((-1, -1), (-1, 1), (1, -1), (1, 1)) and unproject each pixel's clip-space position into world-space spreadsheet coordinates using the inverse view-projection matrix (u_ViewProjectionInvMatrix)[15].

Inside the fragment shader, use hardware screen-space partial derivatives (fwidth(coord) = max(|dfdx|, |dfdy|)) to compute anti-aliased grid lines of constant screen thickness across any zoom level in a single GPU draw call[15]:

```
OpenGL Shading Language
// Fragment shader for zero-geometry uniform grid lines
vec4 render_grid_lines(vec2 worldCoord, vec2 cellSize) {
  // Compute distance to nearest cell boundary in screen pixels using fwidth()
  vec2 grid = abs(fract(worldCoord / cellSize - 0.5) - 0.5) / fwidth(worldCoord) * cellSize;
  float line = min(grid.x, grid.y);
  float alpha = 1.0 - min(line, 1.0);
  return vec4(u_GridColor.rgb, u_GridColor.a * alpha);
}
```

Because GPU fragment shaders evaluate in 2×2 pixel blocks, fwidth(worldCoord) measures the exact world-space derivative per screen pixel, preventing moiré artifacts when zooming out and supporting multi-scale coarse/fine grid hierarchies (gridSize2 = gridSize1 / 5.0)[15]. You can also dynamically adjust grid line contrast against cell background brightness using perceptual luminance weights (vec3(0.299, 0.587, 0.114)) so lines remain legible in both light and dark modes[15].

2. Extruded Line Meshes and 15×30 Spatial Hashing (Variable Cell Sizes)

When rows and columns have custom sizes, procedural fract() math cannot place arbitrary cell boundaries. Because native WebGL line primitives lack cross-platform thickness control, WebGL spreadsheet engines extrude each grid line and border along its normal vector into a thin rectangle made of two triangles[15]. Using PrimitiveTopology.TRIANGLE_STRIP reduces vertex counts from $3N$ vertices for N triangles down to $N + 2$ vertices, while packing RGBA colors into a 4-byte UNSIGNED_BYTE (Format.U8_RGBA_NORM: r | (g << 8) | (b << 16) | (a << 24)) minimizes vertex buffer bandwidth[15].

To avoid rebuilding a massive vertex buffer across the entire viewport whenever a single cell border or dimension changes, Quadratic's WebGL architecture partitions the grid into **15-column by 30-row spatial hashes** (450 cells per hash)[1]. Each spatial hash maintains its own vertex buffers for its **Triangle Shader** (batched cell background fills) and **Line Shader** (batched grid lines and borders)[1]. Editing a cell's border or resizing a row recalculates geometry only for the affected 15×30 spatial hashes, reducing update work from $O(\text{visible cells})$ to $O(\text{cells per hash})$—a reported 100x+ reduction in geometry computation[1]. A dedicated TypeScript Render Worker computes these vertex, UV, index, and color buffers for visible hashes (plus overscan padding) and transfers ownership to the Main Thread via zero-copy Transferable ArrayBuffers (postMessage({ vertices, uvs, indices, colors }, [vertices.buffer, ...])), while evicting distant hashes via an LRU policy when memory exceeds 500 MB[1].

Tier 3: Hyper-Optimized 2D Virtualized DOM

If your architecture requires native DOM elements for cells, you must prevent browser layout

thrashing and compositor bottlenecks[4]. In 1771 Technologies' Playwright/Chrome v149 benchmark on an Apple M4 MacBook Air (testing a 1920x1080 viewport with 0 overscan buffer and a colored div per cell), most React DOM grids degraded as datasets scaled to 1 million rows: AG Grid dropped from 38 FPS at 10K rows to 27 FPS at 1M rows (~29% degradation), MUI X Data Grid Premium dropped from ~60 FPS to 33 FPS (~14% degradation), DevExtreme averaged 17 FPS, and Handsontable averaged ~29 FPS at small scales before crashing with out-of-memory errors above ~150,000 rows[13]. Only LyteNyte Grid sustained 60 FPS across 10K to 1M rows (and 58 FPS with 2 pinned top/bottom rows and 2 pinned left/right columns across 200K rows and 300 columns)[13].

To reach 60 FPS in a DOM-based spreadsheet grid, enforce five engineering constraints:

1. **Two-Axis Virtualization with Bounded Overscan:** Virtualize both rows and columns so only the visible viewport plus a small overscan buffer mounts to the DOM[19]. Tune overscan to 3–5 rows/columns for simple cell geometry (rowBuffer = 5, down from AG Grid's default 10), and enforce a hard safety ceiling (such as AG Grid's 500-row maximum rendered row guard) so misconfigured container dimensions cannot mount 10,000 rows and crash the browser[19].
2. **Row-Level transform: translateY() Positioning:** Never reposition scrolled rows by mutating top and left CSS properties, which triggers synchronous layout reflows[16]. Position virtualized elements using transform: translateY(${rowStart}px) to keep movement on the GPU compositor[16]. Apply transform to **entire row containers** rather than individual cells to avoid exhausting GPU compositor layers[17], or disable row transforms (suppressRowTransform = true) and animations (animateRows = false) entirely when row transitions are unnecessary[41].
3. **DOM Node Pooling and Dirty Checking:** Continuously creating and garbage-collecting nodes during rapid scrolling causes frame drops[2]. Maintain a fixed pool of reusable row and cell DOM nodes; as a row scrolls out of view, recycle its DOM node for the incoming row and dirty-check its coordinates and styles before mutating the DOM[11].
4. **Strict Fixed Pixel Sizing in CSS:** Avoid min-width, percentage widths, or content-driven auto-sizing inside the virtual scroll container[4]. Resolve all row heights and column widths in JavaScript/WASM and apply explicit pixel (px) dimensions so browser layout calculations remain deterministic[4].
5. **requestAnimationFrame Scroll Synchronization:** Throttle scroll handling and batch all DOM reads and coordinate writes inside requestAnimationFrame() to prevent interleaved layout thrashing[4].

2. Spatial Data Structures for Variable Cell Dimensions

Regardless of whether you draw cells via Canvas, WebGL, or DOM, every scroll frame and pointer hit-test must translate between **logical grid indices** (r, c) and **cumulative pixel coordinates** (y, x) [5]. Because row heights and column widths operate on orthogonal 1D axes (except for merged cells, addressed in Section 3), a spreadsheet maintains two independent 1D

spatial indices: one for N rows and one for M columns[5].
Each 1D spatial index must support three operations[5]:

- **offsetOfIndex(i) (Prefix Sum Query):** Returns $\sum_{k=0}^{i-1} \text{size}[k]$, the exact pixel coordinate where row or column i starts.
- **indexAtOffset(px) (Search Query):** Given scrollTop (or pointer coordinate y), finds the largest index i where $\text{offsetOfIndex}(i) \leq \text{px}$.
- **updateSize(i, newSize) (Point or Range Update):** Updates the height of row i (or a range of rows when hiding/unhiding or bulk-resizing) and updates downstream pixel offsets.

Spatial Data Structure	offsetOfIndex(i)	indexAtOffset(px)	updateSize(i, val)	RAM for 1,048,576 Rows	Optimal Spreadsheet Use Case
1. Fixed-Size Math	$O(1)$ (i * size)	$O(1)$ (⌊px / size⌋)	Unsupported (unless global)	**0 Bytes**	Read-only logs or strict uniform grids[5]
2. Dense Prefix-Sum Array	$O(1)$ array lookup	$O(\log N)$ binary search	$O(N)$ array rebuild	**4.00 MB** (Uint32) / **8.00 MB** (Float64)[43]	Static variable heights that rarely resize after initial load[44]
3. Fenwick Tree (BIT)	$O(\log N)$ bit walk	$O(\log N)$ binary lifting	$O(\log N)$ bit walk	**6.00 MB** (Uint32 tree + Uint16 sizes)[43]	High-frequency dynamic resizing of arbitrary individual rows/cols[5]
4. Chunked Prefix Array (B =	$O(B)$ local scan	$O(\log(N/B$	$O(N/B)$ block	**2.02 MB** (Uint32 blocks +	Fast cache-friendly lookups

256)			update	Uint16 sizes)[43]	with moderate resize frequency[43]
5. Sparse RLE / flat_segment_tree	$O(\log K)$ or $O(1)$ w/ hint	$O(\log K)$ binary search	$O(K)$ segment split/merge	**7.81 KB** (for K = 500 custom runs)[43]	**True spreadsheets** (millions of default rows, sparse custom runs)[7]
6. Local Anchor (RowScroller)	$O(W)$ window walk	$O(1)$ (already anchored)	$O(1)$ local map write	**< 4 KB** (stores visible window only)[11]	10M+ row streams where global pixel offset is not required[11]

1. Dense Prefix-Sum Array + Binary Search

Virtualization libraries such as TanStack Virtual and @vielzeug/scroll maintain a cumulative prefix-sum array where offset[i] = offset[i - 1] + height[i - 1][11]. Looking up the pixel offset of row i is an $O(1)$ array read, and finding the first visible row at scrollTop takes $O(\log N)$ via binary search (≤20 comparisons for 1,048,576 rows)[44]. However, dense prefix arrays suffer from an $O(N)$ **update penalty**: if a user drags the border of Row 1 to resize it, every subsequent entry in offset[1 … 1048576] must be recomputed on every pointer-move frame[47].

2. Fenwick Tree (Binary Indexed Tree) in JS or Rust/WASM

To achieve both $O(\log N)$ offset queries and $O(\log N)$ dynamic resize updates, replace the flat prefix-sum array with a **Fenwick Tree (Binary Indexed Tree)**, as implemented in @pdanpdan/virtual-scroll (itemSizesY, itemSizesX, columnSizes) and Rust/WASM virtualizers[5].

A Fenwick tree decomposes any prefix range $[1, N]$ into at most $\log_2 N$ intervals using the lowest set binary bit, lowbit(i) = i & -i, where tree[i] stores the sum of heights in (i - (i & -i), i][47]. When row idx (1-indexed) changes height by delta, point updates propagate upward in ≤20 bitwise steps for 1,048,576 rows, while prefix sum queries accumulate downward in ≤20 steps[47]:

```
TypeScript
// O(log N) point update when a row height or column width changes
function updateDelta(tree: Uint32Array, idx: number, delta: number): void {
  for (let i = idx; i < tree.length; i += i & -i) {
    tree[i] += delta;
  }
}

// O(log N) prefix sum to find exact pixel offset of row/col idx
function queryOffset(tree: Uint32Array, idx: number): number {
  let sum = 0;
  for (let i = idx; i > 0; i -= i & -i) {
    sum += tree[i];
  }
  return sum;
}
```

To find which row intersects scrollTop (indexAtOffset), binary lifting walks down the powers of two from 2^19 (524,288) to 2^0 (1), testing whether tree[idx + bit] <= remainingPx in a single $O(\log N)$ pass[24]. In a developer benchmark comparing a Rust/WASM Fenwick tree virtualizer (set_count, update_height, get_range) against react-window, moving the Fenwick tree math to WASM incurred ~0.1ms overhead per call with zero garbage collection pressure, sustaining 120 FPS at 100K rows, 118 FPS at 1M rows (where react-window dropped to ~12 FPS), and 115 FPS at 10M rows (where react-window crashed)[24].

3. Sparse Run-Length Encoded (RLE) Flat Segment Tree (mdds Pattern)

While a Fenwick tree indexes 1,048,576 rows in 6.00 MB of RAM, allocating multi-megabyte dense arrays per worksheet is unnecessary when 99.9% of rows share the default height[12]. Production spreadsheet engines—such as **LibreOffice Calc** via Kohei Yoshida's Multi-Dimensional Data Structure (mdds) library—use a **flat_segment_tree** to store row heights, column widths, and hidden row/column flags[7].

Instead of storing one entry per row, a flat segment tree stores contiguous runs of identical height in a sorted array of non-overlapping segments[7]:

$$\text{Segment } k = \{\text{startIndex}, \text{endIndex}, \text{unitSize}, \text{cumulativePixelOffset}\}$$

If a 1,048,576-row worksheet has a default 20px row height and the user resizes or hides 500

distinct row ranges (K = 500), the entire sheet's vertical geometry is represented by at most 2K + 1 ≈ 1,001 segments (requiring 7.81 KB for 500 16-byte segments, a 99.9% memory reduction compared to an 8.00 MB dense array)[43]. Given any scrollTop, a binary search across the K segments' cumulativePixelOffset values identifies the active segment s in $O(\log K)$ time, and simple division resolves the exact row index inside that segment in $O(1)$ time:

$$\text{rowIndex} = s.\text{startIndex} + \left\lfloor \frac{\text{scrollTop} - s.\text{cumulativePixelOffset}}{s.\text{unitSize}} \right\rfloor$$

Furthermore, by returning a **position hint** (storing the index of the last accessed segment), sequential row lookups across the visible viewport step through adjacent segments in $O(1)$ **amortized time** without restarting searches from segment 0[46]. Hidden rows and columns are represented naturally as segments with unitSize = 0[7].

3. Spatial Indexing for Merged Cells (rowSpan & colSpan)

Merged cells break the 1D independence of row and column indices[2]. Suppose a merged cell spans rows 5 through 60 and columns 1 through 4 (B6:E61). When the user scrolls down so the viewport displays rows 40 through 90 and columns 3 through 15, the merged cell's top-left anchor (row 5, col 1) lies completely outside the visible viewport. A renderer that only inspects cells whose anchor (r, c) falls inside the visible index window will miss the merged span and erroneously draw standard grid lines through rows 40–60 and columns 3–4[42].

To render merged cell geometry rapidly at scale, follow a four-step spatial pipeline:

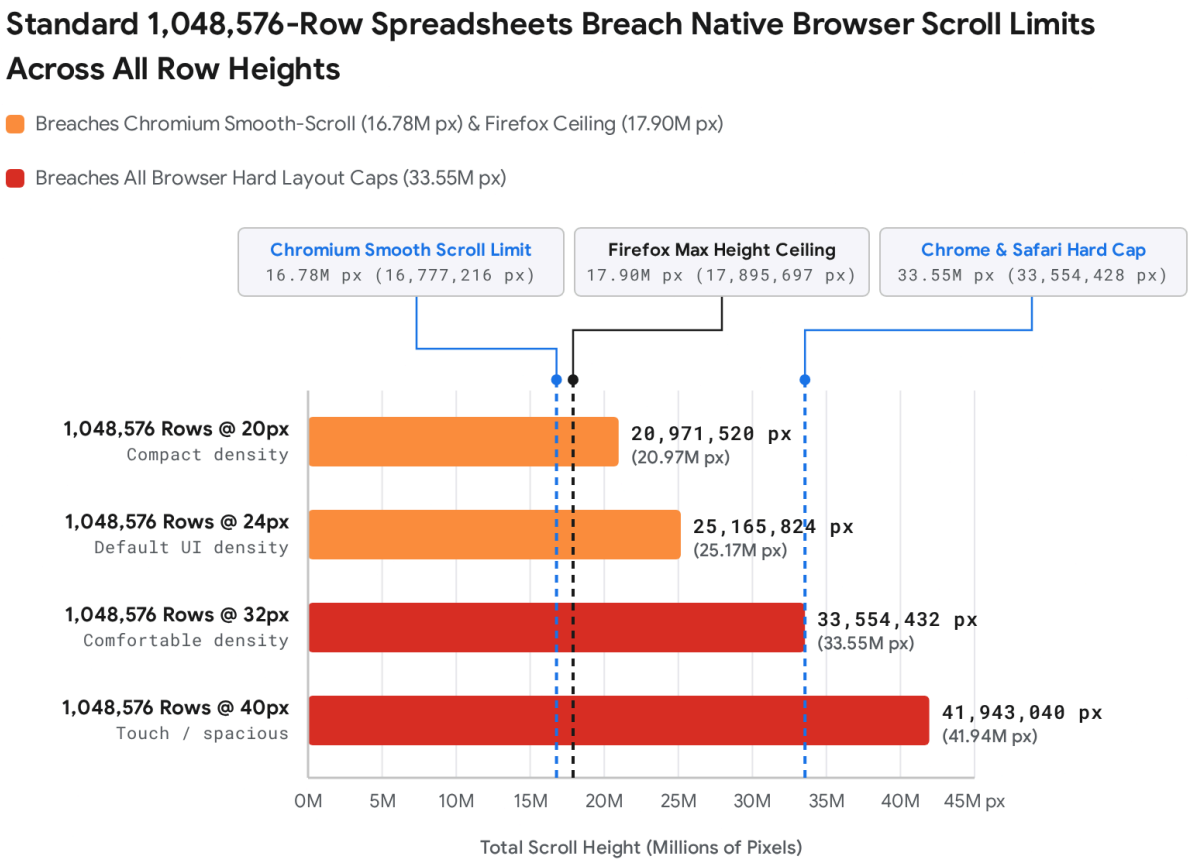
1. **Index Merged Spans in a 2D R-Tree:** Store each merged cell range as a 2D Minimum Bounding Rectangle (MBR) $[c_{\text{start}}, r_{\text{start}}, c_{\text{end}}, r_{\text{end}}]$ inside an **R-tree** (or **R*-tree** such as Rust's rstar or mdds::rectangle_set in LibreOffice Calc)[7].
2. **Query Viewport Intersections:** On each frame, after resolving the visible index window $V = [c_{\min}, r_{\min}, c_{\max}, r_{\max}]$ from your 1D row and column trees, query the R-tree for all merged rectangles intersecting V in $O(\log M_{\text{merges}} + k)$ time[50].
3. **Resolve Pixel Bounding Boxes:** For each intersecting merge $[c_1, r_1, c_2, r_2]$, query your 1D spatial trees to compute its viewport-relative rectangle: x = colTree.offsetOfIndex(c1) - scrollLeft, y = rowTree.offsetOfIndex(r1) - scrollTop, width = colTree.offsetOfIndex(c2 + 1) - colTree.offsetOfIndex(c1), and height = rowTree.offsetOfIndex(r2 + 1) - rowTree.offsetOfIndex(r1)[5].

4. **Overpaint Interior Grid Lines (Painter's Algorithm):** Rather than fragmenting your fast batched beginPath() grid-line pass around merged regions, stroke the continuous viewport grid lines first, then iterate over the k visible merged rectangles and call ctx.fillRect(x, y, width, height) (using the merged cell's background fill or default sheet background) followed by ctx.strokeRect(x, y, width, height) to mask interior lines and draw the merged border.

4. Overcoming Browser Scroll Container Pixel Limits

Even with a 60 FPS renderer and an $O(\log N)$ spatial index, standard virtual scrolling fails when total sheet height exceeds browser CSS coordinate ceilings[8].
Conventional virtual scrollers create a native scrollbar by placing a spacer inside an overflow: auto container sized to totalRows * rowHeight[9]. Because browser layout engines store coordinates using fixed-point representations (such as Chromium's LayoutUnit), they truncate element dimensions and clamp scroll offsets above hard pixel limits documented in browser bug trackers and developer measurements[8]:

- **Chromium Smooth-Scroll / Culling Limit:** 16,777,216 px (2^24 px)—above this threshold in Chromium, slow scrolling stalls or fails to advance content properly[8].
- **Firefox Maximum Element Height:** ~17,000,000 to 17,895,697 px—Firefox truncates element heights above roughly 17 million pixels[10].
- **Chrome & Safari Hard Layout Ceiling:** 33,554,428 px (2^25 - 4 px)—the maximum height, width, scrollHeight, or scrollWidth accepted by Chrome and Safari[8].



Total vertical height required to scroll 1,048,576 rows (Excel worksheet row capacity) at 20px, 24px, 32px, and 40px row heights. Even at 20px per row (20.97M px), a 1M-row sheet exceeds Chromium's 16.78M px smooth-scroll limit and Firefox's 17.90M px layout ceiling, requiring scrollbar downscaling, runway recentering, or custom scroll panes. Notice that at 32px per row (33,554,432 px), the grid exceeds Chrome and Safari's 33,554,428 px layout cap by 4 pixels.

Data sources: Fast.io, Chromium Issue Tracker, DEV Community (Kohii), RedNegra

At 1,048,576 rows, even a compact 20px row height requires 20,971,520 px, breaching both Chromium's 16.77M px scroll limit and Firefox's ~17M px ceiling[8]. At 32px (33,554,432 px) or 40px (41,943,040 px), native scroll containers fail in every browser[8]. To scroll millions of rows without truncation, implement one of three scroll architectures:

Pattern A: Hybrid Scrollbar Downscaling (HighTable & SlickGrid)

To retain a native browser scrollbar, cap the dummy scroll container height (maxCanvasHeight) to a safe constant such as 8,000,000 px and compute a **Downscale Factor** when logical height (fullTableHeight) exceeds that threshold[10]:

$$\text{downscaleFactor} = \frac{\text{fullTableHeight} - \text{viewportHeight}}{\text{maxCanvasHeight} - \text{viewportHeight}}$$

Pure linear downscaling creates a precision gap at extreme scales: when a 1px native scroll step maps to thousands of logical pixels, intermediate rows become unreachable[10]. HighTable's hybrid technique and SlickGrid's paged scroll resolve this by tracking the true logical offset in 64-bit JavaScript floats and branching on scroll delta magnitude: small scroll movements (mouse wheel, trackpad, or keyboard steps) apply 1:1 local pixel deltas so every row is reachable, whereas large scroll movements (dragging the scrollbar thumb) scale by downscaleFactor to jump globally across the dataset[10].

Pattern B: Fixed-Height Native Runway with Sticky Recentering (RowScroller)

When global pixel coordinates are unnecessary, RowScroller tracks scroll state purely as an index and local pixel offset ({ rowIndex: number, offsetWithinRow: number }) inside a fixed 20,000 px native scroll runway housing a sticky viewport (position: sticky; top: 0; overflow: clip;)[11].

As the user scrolls, native scroll deltas (currentScrollTop - lastScrollTop) are added to offsetWithinRow; whenever offsetWithinRow passes the current row's height, rowIndex increments (or decrements on upward scroll) and the visible window is laid out by walking forward from rowIndex starting at -offsetWithinRow[11]. Whenever currentScrollTop comes within 4,000 px of either end of the 20,000px runway, the component silently resets scrollTop to the center ((20000 - containerHeight) / 2) and updates lastScrollTop to match[11]. Because the sticky viewport does not move when scrollTop recenters, native inertial scrolling works seamlessly across 10 million dynamic-height rows while keeping browser CSS coordinates under 20,000px[11].

Pattern C: Custom Stateless ScrollPane with Relative Offsets

Used by full Canvas spreadsheets like Univer, this pattern sets overflow: hidden on the viewport container and renders a custom ScrollBar (Track and Thumb) alongside the canvas[3].

Given viewportSize and total contentSize, the custom scrollbar enforces a minimum clickable thumb height via thumbSize = Math.max(16, (viewportSize / contentSize) * viewportSize) and maps thumb position linearly to logical scrollPosition[9]. Wheel events (event.deltaY) and thumb drags update scrollPosition directly in JavaScript memory[9]. During rendering, a row at logical offset startPosition (even if startPosition is 90,000,000 px) is drawn at top = startPosition - scrollPosition, keeping all rendered coordinates small and viewport-relative[9].

5. Architectural Decision Matrix

To select the right combination of rendering backend, spatial index, and scroll mechanics for your target workload, match your requirements against the three production profiles below:

Target Spreadsheet Profile	Recommended Rendering Backend	Spatial Coordinate Index	Scroll Virtualization Mechanism

1. Enterprise Data Grid (<500K rows, <200 cols, frequent individual row/col resizing)	**HTML5 2D Canvas** (batched beginPath grid lines snapped to ⌊coord * dpr⌋ + 0.5) or **Pooled 2D DOM** (row-level translateY)[2]	**Fenwick Tree (BIT)** in Uint32Array (6.00 MB for 1M rows; $O(\log N)$ resize and offset lookup)[5]	**Hybrid Downscaled Native Scrollbar** capped at 8,000,000 px (downscaleFactor + 1:1 local wheel deltas)[10]
2. Full Cloud Spreadsheet (1M–20M cells, merged cells, frozen panes, mostly uniform rows)	**Layered HTML5 2D Canvas / OffscreenCanvas** in a Web Worker with ctx.clip() quadrant scissor rects[2]	**Sparse RLE flat_segment_tree** with position hints (<10 KB for 500 custom runs) + **2D R-Tree** for merged cells[7]	**Custom Stateless ScrollPane** (overflow: hidden, 16px min thumb, relative startPosition - scrollPosition)[9]
3. Infinite-Canvas / Analytical Grid (10M+ cells, continuous 0.01x–10x camera zoom)	**WebGL / WebGPU** with fwidth() fragment shader for uniform grids + **15×30 cell spatial hashes** via zero-copy Transferable buffers[1]	**Rust/WASM Fenwick or Sparse Segment Tree** synced with Render Worker via SharedArrayBuffer[1]	**2D Camera Matrix Viewport** (u_ViewProjectionInvMatrix) with 500 MB LRU spatial hash eviction[1]

Works cited

1. Building a High-Performance Spreadsheet Renderer - Quadratic AI, https://www.quadratichq.com/blog/building-a-high-performance-spreadsheet-renderer-why-we-chose-webgl-over-html
2. https://prompts.brightcoding.dev/blog/glide-data-grid-the-revolutionary-canvas-grid-for-react-apps
3. https://docs.univer.ai/blog/this-is-univer
4. Rendering Massive Tables at Lightning Speed: Virtualization with, https://dev.to/lalitkhu/rendering-massive-tables-at-lightning-speed-virtualization-with-virtual-scrolling-2dpp
5. Documentation | Virtual Scroll - PDan Blog, https://pdanpdan.github.io/virtual-scroll/docs/
6. Drawing crisp 1px lines on HTML5 canvas when using transformations, https://stackoverflow.com/questions/53288682/drawing-crisp-1px-lines-on-html5-canvas-when-using-transformations

7. Vision for Calc Development Keeping Calc Modular Kohei Yoshida, https://kohei.us/files/slides_libocon_2011.pdf
8. Maximum element size is too restrictive for virtual scrolling [41441393], https://issues.chromium.org/41441393
9. How to Implement Virtual Scrolling Beyond the Browser's Limit, https://dev.to/kohii/how-to-implement-virtual-scrolling-beyond-the-browsers-limit-16ol
10. Virtual Scrolling for Billions of Rows — Techniques from HighTable, https://rednegra.net/blog/20260212-virtual-scroll/
11. Virtual rendering for massive lists - Brian Simon, https://brianesimon.com/blog/virtual-rendering-massive-lists
12. Google Sheets Row Limit: Max Rows, Cell Caps & Fixes | Fastio, https://fast.io/resources/google-sheets-row-limit/
13. https://www.1771technologies.com/blog/performance-benchmarks
14. OffscreenCanvas: Browser Support, Features, Known Issues, https://www.testmuai.com/learning-hub/offscreencanvas-browser-support/
15. Lesson 5 - Grid | An infinite canvas tutorial, https://infinitecanvas.cc/guide/lesson-005
16. Build your Own Virtual Scroll - Part I - DEV Community, https://dev.to/adamklein/build-your-own-virtual-scroll-part-i-11ib
17. https://angular.love/inside-ag-grid-techniques-to-make-the-fastest-javascript-datagrid-in-the-world
18. Data Grid Virtual Scrolling in JavaScript Explained - Simple Table, https://www.simple-table.com/blog/data-grid-virtual-scrolling-javascript
19. JavaScript Grid: DOM Virtualisation, https://www.ag-grid.com/javascript-data-grid/dom-virtualisation/
20. JavaScript Grid: Legacy Themes: Legacy Sass API - AG Grid, https://www.ag-grid.com/javascript-data-grid/theming-v32-customisation-sass-legacy/
21. HTML Canvas Tutorial: 7 Interactive Beginner Examples | Savvy, https://savvy.co.il/en/blog/wordpress-development/html-canvas-beginners-guide/
22. Dirty-Rectangle Rendering With Apache Echarts - YouTube, https://www.youtube.com/watch?v=H_gC9TLzlUo
23. HTML5 Canvas drawings like lines are looking blurry - TutorialsPoint, https://www.tutorialspoint.com/article/html5-canvas-drawings-like-lines-are-looking-blurry
24. I moved virtualization math to Rust/WASM - here's what I learned, https://www.reddit.com/r/reactjs/comments/1q5kt5u/i_moved_virtualization_math_to_rustwasm_heres/
25. Drawing Sharp Lines on the Canvas - KIRUPA, https://www.kirupa.com/canvas/drawing_sharp_lines_canvas.htm
26. HTML5 Canvas – Crisp lines every time - mobtowers - WordPress.com, https://mobtowers.wordpress.com/2013/04/15/html5-canvas-crisp-lines-every-time/

27. Complete Solution for Drawing 1 Pixel Line on HTML5 Canvas, https://stackoverflow.com/questions/25277023/complete-solution-for-drawing-1-pixel-line-on-html5-canvas
28. Device Pixel Ratio - Chart.js, https://www.chartjs.org/docs/latest/configuration/device-pixel-ratio.html
29. How to fix a canvas so it will look good on retina/high-DPI screens., https://gist.github.com/callumlocke/cc258a193839691f60dd?permalink_comment_id=4068338
30. Drawing shapes with canvas - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Drawing_shapes
31. Architecture of Rendering Engine | Univer Office SDK, https://docs.univer.ai/blog/rendering
32. 渲染引擎架构设计 - Univer Office SDK, http://docs.univer.ai/zh-CN/blog/rendering
33. OffscreenCanvas and worker threads, https://mdn.github.io/dom-examples/web-workers/offscreen-canvas-worker/
34. OffscreenCanvas—speed up your canvas operations with a web, https://web.dev/articles/offscreen-canvas
35. OffscreenCanvas - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/OffscreenCanvas
36. Lesson 5 - Grid | An infinite canvas tutorial - AntV, https://antv.vision/infinite-canvas-tutorial/guide/lesson-005
37. Optimal way of rendering lines - Unity Discussions, https://discussions.unity.com/t/optimal-way-of-rendering-lines/761577
38. Fissures WebGL Demo - Anti-Aliased Grid Shader, https://thorium.rocks/media/grid/fissures.html
39. Instanced Lines for Vector Maps | Jeshurun Hembd | Observable, https://observablehq.com/@jjhembd/instanced-lines-for-vector-maps
40. Frontend System Design: Virtualization & Handling Large Data Sets, https://dev.to/zeeshanali0704/frontend-system-design-virtualization-handling-large-data-sets-29nf
41. AG Grid Performance Hacks: Making Data Grids Lightning Fast, https://medium.com/@tech-stack/ag-grid-performance-hacks-making-data-grids-lightning-fast-1e120dadb487
42. Virtual Scrolling in EJ2 JavaScript Data Grid - Syncfusion, https://ej2.syncfusion.com/javascript/documentation/grid/scrolling/virtual-scrolling
43. unknown_url
44. Architecture & Competitive Position — vlist v1 docs, https://vlist.io/docs/v1/resources/roadmap
45. Scroll: Virtual list engine for TypeScript - Vielzeug, https://vielzeug.dev/scroll/
46. Tag: mdds - Roundtrip to Shanghai via Tokyo, https://kohei.us/tag/mdds/
47. Fenwick or Binary Index Tree - Dev Genius, https://blog.devgenius.io/fenwick-or-binary-index-tree-959c75a2000a
48. Author: Kohei Yoshida - Roundtrip to Shanghai via Tokyo,

https://kohei.us/author/admin/
49. libreoffice-impress-26.2.5.1-bp161.1.5 RPM for x86_64 - RPMfind.net, https://www.rpmfind.net/linux/RPM/opensuse/16.1/x86_64/libreoffice-impress-26.2.5.1-bp161.1.5.x86_64.html
50. What Is an R-Tree? A Complete Guide to Spatial Indexing - Lizardtech, https://www.lizardtech.com/post/what-is-an-r-tree-a-complete-guide-to-spatial-indexing
51. Spatial Index: R Trees - Towards Data Science, https://towardsdatascience.com/spatial-index-r-trees-5ac6ad36ca20/
52. Algorithms — list of Rust libraries/crates // Lib.rs, https://lib.rs/algorithms
53. Repo Fixes · 6pac SlickGrid · Discussion #1247 · GitHub, https://github.com/6pac/SlickGrid/discussions/1247
54. slickgrid - UNPKG, https://app.unpkg.com/slickgrid@5.9.0/files/dist/browser/slick.grid.js.map
55. jQuery Grid Scroll Modes Virtual Scrolling - Telerik.com, https://www.telerik.com/kendo-jquery-ui/documentation/controls/grid/scrolling/virtual-scrolling